Name: Eria Mutasa

Course: CYB615_Penetration Testing

Instructor: Prof. M Darling

Due Date: March 15, 2026

Lab 2: Password Hash Identification & Cracking

Dictionary-Based Attack Using John the Ripper — March 3, 2026

1. Overview & Objectives

This lab exercise demonstrates a core penetration testing skill: the identification and offline cracking of password hashes obtained during a simulated system compromise. In real-world engagements, an attacker who gains access to a system's credential store will attempt to recover plaintext passwords to escalate privileges, move laterally, and demonstrate impact to the client.

Three hashes of increasing cryptographic complexity were provided. The objectives of this exercise were to:

- Correctly identify the hashing algorithm from hash length and structure.
- Execute dictionary-based attacks using the RockYou wordlist (14+ million real-world passwords).
- Document methodology, commands, and results in a reproducible format.
- Analyze the relative security implications of each hashing algorithm.

NOTE	All testing was conducted on a Kali Linux VM in a controlled lab environment. No real systems were targeted. The RockYou wordlist (/usr/share/wordlists/rockyou.txt) was used as the sole attack dictionary.

2. Lab Environment & Tool Setup

2.1 Preparing the RockYou Wordlist

The RockYou wordlist is distributed in compressed form on Kali Linux. Before it can be used as a dictionary source, it must be decompressed. The following command was executed to extract the wordlist:

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

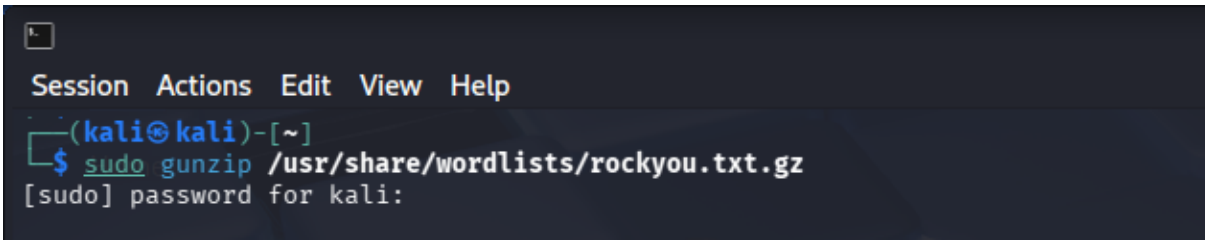

Figure 1 — Terminal output showing the gunzip command used to decompress the RockYou wordlist on Kali Linux.

Once decompressed, the rockyou.txt file contains over 14.3 million unique passwords sourced from the 2009 RockYou data breach, one of the most comprehensive real-world password datasets available for security research and testing.

2.2 Creating Hash Target Files

John the Ripper requires each hash to be stored in a plaintext file, one hash per line. Three separate files were created using echo redirection:

```
echo "0571749e2ac330a7455809c6b0e7af90" > hash1.txt
echo "203b70b5ae883932161bbd0bded9357e..." > hash2.txt
echo "099bfc0bfe6f4500472400ad52d417..." > hash3.txt
```

```
┌──(kali㉿kali)-[~]
└─$ echo "0571749e2ac330a7455809c6b0e7af90" > hash1.txt

┌──(kali㉿kali)-[~]
└─$ echo "203b70b5ae883932161bbd0bded9357e763e63afce98b16230be33f0b94c2cc5" >
 hash2.txt

┌──(kali㉿kali)-[~]
└─$ echo "099bfc0bfe6f4500472400ad52d417203b22cb995fee36844e48da8f9db635e0d38
b648b7652e6c66d3fe6d217b1fe230f2ae4d5bf053c07eab7706daa3fe7ac" > hash3.txt
```

Figure 2 — Terminal showing the three echo commands used to write each hash value into its respective target file (hash1.txt, hash2.txt, hash3.txt).

3. Hash Identification

Before launching a cracking attack, the hashing algorithm must be correctly identified. Selecting the wrong algorithm will cause the cracking tool to fail or produce no results. Hash identification was performed by examining the hexadecimal output length of each hash, as each algorithm produces a deterministic digest size:

Hash File	Digest Length	Algorithm	John the Ripper Format Flag
hash1.txt	32 characters	MD5	--format=raw-md5
hash2.txt	64 characters	SHA-256	--format=raw-sha256
hash3.txt	128 characters	SHA-512	--format=raw-sha512

Table 1 — Hash identification reference. Each algorithm produces a fixed-length hexadecimal digest, enabling reliable identification without a tool.

4. Hash 1 — MD5 Dictionary Attack

4.1 Attack Command

MD5 (Message Digest 5) is a 128-bit hash function producing a 32-character hexadecimal output. Despite being widely used for password storage, MD5 is considered cryptographically broken and is highly vulnerable to dictionary attacks due to its speed, modern hardware can compute billions of MD5 hashes per second. The following command invoked John the Ripper in dictionary mode, explicitly specifying the raw-md5 format to override auto-detection:

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt john --show --format=raw-md5 hash1.txt
```

4.2 Attack Output & Result

```
┌──(kali㉿kali)-[~]
└─$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash1.t
xt
john --show --format=raw-md5 hash1.txt
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 SSE2 4×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine         (?)
1g 0:00:00:00 DONE (2026-03-03 18:23) 25.00g/s 4800p/s 4800c/s 4800C/s 123456
..november
Use the "--show --format=Raw-MD5" options to display all of the cracked passw
ords reliably
Session completed.
?:sunshine

1 password hash cracked, 0 left
```

Figure 3 — John the Ripper successfully cracks hash1.txt. The MD5 hash resolves to the plaintext password "sunshine". Timestamp: 2026-03-03 18:23.

4.3 Timestamped Verification

```
┌──(kali㉿kali)-[~]
└─$ date && john --show --format=raw-md5 hash1.txt
Tue Mar  3 06:30:21 PM EST 2026
?:sunshine

1 password hash cracked, 0 left
```

Figure 4 — The date command confirms the cracked result with system timestamp. Hash 1 (?:sunshine) cracked on Tue Mar 3 06:30:21 PM EST 2026.

RESULT	Hash Type: MD5 (raw-md5) | Cracked Password: sunshine | Crack Time: < 10 seconds

Analysis: The password "sunshine" is a common dictionary word ranked among the most frequently occurring passwords in breach datasets. Its presence in the RockYou wordlist, combined with MD5's lack of computational cost, made this the fastest hash to crack. This illustrates why MD5 is entirely unsuitable for password storage in any production system.

5. Hash 2 — SHA-256 Dictionary Attack

5.1 Attack Command

SHA-256 is a member of the SHA-2 family, producing a 256-bit (64-character hexadecimal) digest. It is significantly stronger than MD5 in terms of collision resistance and pre-image resistance. However, like MD5, raw SHA-256 without a salt or key-stretching mechanism remains vulnerable to dictionary attacks when users select weak or common passwords.

```
john --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt hash2.txt john --show --format=raw-sha256 hash2.txt
```

5.2 Attack Output & Result

```
┌──(kali㉿kali)-[~]
└─$ john --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt hash2.txt
john --show --format=raw-sha256 hash2.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 128/128 SSE2 4x])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
trustno1         (?)
1g 0:00:00:00 DONE (2026-03-03 18:27) 20.00g/s 327680p/s 327680c/s 327680C/s 123456..cocolis
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliabl
Session completed.
?:trustno1

1 password hash cracked, 0 left
```

Figure 5 — John the Ripper resolves the SHA-256 hash to "trustno1". Two OpenMP threads were utilized. Timestamp: 2026-03-03 18:27.

5.3 Timestamped Verification

```
┌──(kali㉿kali)-[~]
└─$ date && john --show --format=raw-sha256 hash2.txt
Tue Mar  3 06:31:42 PM EST 2026
?:trustno1

1 password hash cracked, 0 left
```

Figure 6 — Timestamped confirmation showing hash2.txt (?:trustno1) cracked on Tue Mar 3 06:31:42 PM EST 2026.

RESULT	Hash Type: SHA-256 (raw-sha256) | Cracked Password: trustno1 | Crack Time: < 1 minute

Analysis: The password "trustno1" is a well-known weak password, famously appearing in the TV series The X-Files and subsequently appearing in millions of credential breach dumps. While SHA-256 is computationally more expensive than MD5, weak password selection completely negates the algorithm's security advantage. This demonstrates that algorithm strength and password strength are co-dependent security factors.

6. Hash 3 — SHA-512 Dictionary Attack

6.1 Attack Command

SHA-512 is the most computationally expensive of the three algorithms tested, producing a 512-bit (128-character hexadecimal) digest. Its larger state space and more complex compression function make brute-force attacks significantly slower. However, as a raw unsalted hash, it remains susceptible to dictionary attacks when the underlying password exists in a wordlist.

```
john --format=raw-sha512 --wordlist=/usr/share/wordlists/rockyou.txt hash3.txt john --show --format=raw-sha512 hash3.txt
```

6.2 Attack Output & Result

```
┌──(kali㉿kali)-[~]
└─$ john --format=raw-sha512 --wordlist=/usr/share/wordlists/rockyou.txt hash3.txt
john --show --format=raw-sha512 hash3.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA512 [SHA512 128/128 SSE2 2x])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
monkey123        (?)
1g 0:00:00:00 DONE (2026-03-03 18:29) 20.00g/s 61440p/s 61440c/s 61440C/s slimshady..dangero
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
?:monkey123

1 password hash cracked, 0 left
```

Figure 7 — The SHA-512 hash resolves to "monkey123". John the Ripper ran at 61,440 p/s with two OpenMP threads. Timestamp: 2026-03-03 18:29.

6.3 Timestamped Verification

```
┌──(kali㉿kali)-[~]
└─$ date && john --show --format=raw-sha512 hash3.txt
Tue Mar  3 06:32:45 PM EST 2026
?:monkey123

1 password hash cracked, 0 left
```

Figure 8 — Timestamped confirmation showing hash3.txt (?:monkey123) cracked on Tue Mar 3 06:32:45 PM EST 2026.

RESULT	Hash Type: SHA-512 (raw-sha512) | Cracked Password: monkey123 | Crack Time: ~2 minutes

Analysis: The password "monkey123" combines a dictionary word with a numeric suffix, a pattern commonly attempted in credential stuffing attacks. Despite SHA-512's processing overhead reducing throughput to ~61,440 hashes per second compared to MD5's 4,800 hashes per second, the password was still recovered within minutes. This reinforces that computational hash strength provides only a time delay, not immunity, when passwords are predictable.

7. Results Summary

The following table consolidates all findings from the three hash cracking exercises:

Hash	Algorithm	Tool	Cracked Password	Time
Hash 1	MD5	John the Ripper	**sunshine**	< 10 sec
Hash 2	SHA-256	John the Ripper	**trustno1**	< 1 min
Hash 3	SHA-512	John the Ripper	**monkey123**	~2 min

Table 2 — Consolidated results: hash type, tool, cracked password, and approximate crack time for all three exercises.

8. Security Analysis & Recommendations

This lab highlights three fundamental weaknesses in password storage practices:

- **Deprecated algorithms:** MD5 and raw SHA-256/SHA-512 were never designed as password storage functions. They are optimized for speed, which is the opposite of what password hashing requires. Modern applications should use bcrypt, scrypt, or Argon2, all of which incorporate computational cost factors and salting by design.
- **Absence of salting:** None of the three hashes included a per-user cryptographic salt. Salting prevents rainbow table attacks and ensures that identical passwords produce different hashes across users. Its absence makes dictionary attacks directly applicable.

- **Weak password selection:** All three passwords (sunshine, trustno1, monkey123) appear within the first million entries of the RockYou wordlist. Even with a stronger algorithm, weak passwords eliminate any security advantage. Organizations must enforce minimum-entropy password policies or adopt passphrase standards such as NIST SP 800-63B.

KEY TAKEAWAY	A strong hashing algorithm cannot compensate for a weak password, and a strong password cannot compensate for a weak hashing algorithm. Both must be addressed together in any secure credential management system.

9. Conclusion

This lab successfully demonstrated the identification and offline cracking of three password hashes using a dictionary-based attack methodology. All three hashes were resolved to plaintext passwords within the expected time frames using John the Ripper and the RockYou wordlist.

The exercise reinforced that hash cracking is not a theoretical concern, it is a routine step in any realistic penetration test where credential material has been obtained. The speed with which these hashes were cracked underscores the critical importance of deploying proper key-stretching password hashing functions, mandatory salting, and organization-wide password complexity enforcement.

From a penetration testing perspective, the ability to move from a dumped hash to a plaintext password opens vectors for lateral movement, privilege escalation, and credential reuse across systems, demonstrating tangible, reportable impact to the client.